

SINGLE-CYCLE RISC-V PROCESSOR

RV32I Architecture Implementation

A Digital Design and Verification Project

Parameter	Detail
Hardware Description Language	Verilog HDL
Processor Architecture	Single-Cycle RISC-V (RV32I Base Integer ISA)
Instruction Width	32-bit
System Clock	100 MHz (Basys 3 onboard oscillator)
Simulation Tool	Xilinx Vivado Simulator
Target FPGA	Digilent Basys3 (Xilinx Artix-7)
Instruction Memory	256 x 32-bit ROM (loaded via \$readmemh)
Data Memory	128 x 32-bit Block RAM
Register File	32 x 32-bit General Purpose Registers (x0-x31)

Submitted by

K. Venkata Ganesh Koushal (24EG104A13)

G. Anirudh (24EG104A59)

K. Moursagna Rao (24EG104A38)

Department of Electronics & Communication Engineering
Anurag University, Hyderabad

VLSI-CAD Summer Internship 2026

Department of Electronics and Communication Engineering, NIT Rourkela

Table of Contents

1. Summary (Quick Overview)

2. Introduction

2.1 Objective

2.2 Tools and Hardware Used

2.3 Why a Tick Generator Instead of Running at Full Clock Speed

3. Processor Architecture / Datapath Overview

4. Design Code — Top-Level Module (top_riscv.v)

5. Design Code — Program Counter (pc.v)

6. Design Code — Instruction Memory (instruction_memory.v)

7. Design Code — Control Unit (control_unit.v)

8. Design Code — Immediate Generator (imm_gen.v)

9. Design Code — Register File (register_file.v)

10. Design Code — ALU Control (alu_control.v)

11. Design Code — Arithmetic Logic Unit (alu.v)

12. Design Code — Branch Unit (branch_unit.v)

13. Design Code — Data Memory (data_memory.v)

14. Datapath Signal Flow Summary

15. Supported Instruction Set (RV32I)

15.1 R-Type Instructions (Register-Register)

15.2 I-Type Instructions (ALU-Immediate, Loads, JALR)

15.3 S-Type Instructions (Stores)

15.4 B-Type Instructions (Conditional Branches)

15.5 U-Type and J-Type Instructions

16. Test Program (program.mem)

17. Constraints File (riscv.xdc)

18. Testbench (tb_riscv.v)

19. FPGA Implementation Flow — Basys3 Board

20. Results and Outputs

20.1 RTL Schematic Output

20.2 Synthesis Schematic Output

20.3 Simulation Output

20.4 FPGA Hardware Output

21. Conclusion

1. Summary (Quick Overview)

This project implements a Single-Cycle RISC-V Processor, conforming to the base integer instruction set (RV32I), entirely in Verilog HDL. A single-cycle processor executes one complete instruction — fetch, decode, execute, memory access, and write-back — within a single clock cycle. It is the simplest possible CPU microarchitecture and forms the conceptual foundation for understanding pipelined and superscalar processor designs used in real-world CPUs.

In simple terms: the processor reads a pre-loaded program from an internal instruction memory, executes each instruction in sequence (or jumps/branches as instructed), updates its 32 general-purpose registers and data memory accordingly, and exposes selected internal signals (program counter, current instruction, ALU result, write-back data, memory contents, and specific register values) on the Basys3 board's 16 LEDs, selectable using 3 onboard switches.

The design follows the classical five-stage single-cycle datapath, fully unrolled into combinational and sequential logic within one clock cycle:

- Instruction Fetch (IF) — Program Counter (PC) addresses the Instruction Memory (ROM)
- Instruction Decode (ID) — Control Unit decodes the opcode; Register File supplies operands; Immediate Generator extracts and sign-extends immediates
- Execute (EX) — ALU performs the arithmetic/logic operation selected by the ALU Control unit; Branch Unit evaluates branch conditions
- Memory Access (MEM) — Data Memory is read from or written to, for load/store instructions
- Write Back (WB) — Result is multiplexed back into the Register File

The design is broken into nine small, single-responsibility modules, each independently understandable and verifiable:

- `top_riscv.v` — top-level module instantiating and connecting every sub-module, plus a tick generator for human-observable execution speed and an LED debug multiplexer
- `pc.v` — the Program Counter register, advanced once per tick
- `instruction_memory.v` — 256-word instruction ROM, initialized from `program.mem`
- `control_unit.v` — decodes the 7-bit opcode into all datapath control signals
- `imm_gen.v` — extracts and sign-extends immediates for I, S, B, U and J type instructions
- `register_file.v` — 32 x 32-bit general purpose register array (x0 hardwired to zero)
- `alu_control.v` — derives the precise 4-bit ALU operation from `alu_op`, `funct3`, and `funct7`
- `alu.v` — the 32-bit Arithmetic Logic Unit supporting 10 operations
- `data_memory.v` — 128-word byte-addressable data RAM supporting byte/half-word/word load and store
- `branch_unit.v` — evaluates all six RISC-V branch conditions (BEQ, BNE, BLT, BGE, BLTU, BGEU)

The processor was first verified through behavioural simulation in Vivado using a self-checking, instruction-trace testbench, and was subsequently synthesized, implemented, and programmed onto the Basys3 FPGA board for real hardware validation, with execution state observed live on the LED bank.

2. Introduction

RISC-V (“Reduced Instruction Set Computer – Five”) is a free and open instruction set architecture (ISA) originally developed at UC Berkeley. Unlike proprietary ISAs such as x86 or ARM, RISC-V is openly licensed, allowing anyone to implement, modify, and extend it without licensing fees. This has made it the architecture of choice for academic processor design and a rapidly growing presence in industry, from embedded microcontrollers to data-centre accelerators.

RISC-V is built around a small, orthogonal base integer instruction set (RV32I for the 32-bit variant), which can be optionally extended with modules for multiplication/division (M), atomics (A), floating point (F/D), and compressed instructions (C), among others. This project implements the unextended RV32I base integer set.

A Single-Cycle implementation is the most direct hardware realization of an ISA: every instruction, regardless of type, completes in exactly one clock cycle. While this makes the design simple to understand and debug, it is inherently inefficient — the clock period must be long enough to accommodate the slowest instruction (typically a load, which must traverse the ALU and Data Memory in series), wasting time on faster instructions. This trade-off is precisely why real-world processors move to pipelined or out-of-order microarchitectures, which this single-cycle design serves as the conceptual stepping stone toward.

2.1 Objective

- Design a fully functional single-cycle RISC-V (RV32I) processor in Verilog HDL
- Implement all major RV32I instruction formats: R-type, I-type, S-type, B-type, U-type, and J-type
- Verify correct datapath and control behaviour through instruction-level simulation in Vivado
- Implement and test the design on a real Basys3 FPGA board, with live execution state visible on LEDs
- Build a clean, modular RTL structure separating datapath, control, and memory elements for maintainability and ease of verification

2.2 Tools and Hardware Used

Item	Description
Hardware Description Language	Verilog HDL
Architecture / Protocol	Single-Cycle RISC-V — RV32I Base Integer ISA
Simulation / Synthesis Tool	Xilinx Vivado
FPGA Board	Digilent Basys3 (Xilinx Artix-7)
Board Clock Frequency	100 MHz
Instruction Width	32 bits
Register File	32 x 32-bit (x0–x31), x0 hardwired to 0

Item	Description
Instruction Memory	256 words (1 KB), loaded from program.mem via \$readmemh
Data Memory	128 words (512 bytes), inferred as Block RAM
User Interface	3 slide switches (debug select) + 16 LEDs (debug output)

2.3 Why a Tick Generator Instead of Running at Full Clock Speed

Although the processor itself is purely combinational-plus-register single-cycle logic running off the 100 MHz system clock, at full speed it would execute roughly 100 million instructions per second — far too fast for the human eye to observe via LEDs. To make execution observable on hardware, the top module includes a tick generator: a 29-bit counter that produces a single tick pulse roughly every 3 seconds (TICK_COUNT = 300,000,000 cycles at 100 MHz). The Program Counter and Register File only advance on this tick, effectively single-stepping the processor once every 3 seconds so that the selected debug value can be read off the LEDs by eye. In simulation, the testbench overrides this parameter to a much smaller value (TICK_COUNT = 10) so the entire program executes within microseconds.

4. Design Code — Top-Level Module (top_riscv.v)

The top module instantiates and wires together all nine sub-modules into the complete single-cycle datapath, generates the human-observable tick signal, and multiplexes internal debug signals onto the 16-LED output bank based on the 3 input switches.

- Tick generator: divides the 100 MHz clock down to one pulse approximately every 3 seconds, so PC, register file and memory writes advance once per tick — making execution observable on real hardware.
- PC + PC-plus-4 logic: maintains the current instruction address and the default next address.
- Instruction decode field extraction: opcode, rd, funct3, rs1, rs2, funct7 sliced directly from the fetched instruction.
- Write-back multiplexer: selects between ALU result, memory read data, PC+4 (for JAL/JALR), immediate (LUI) or PC+immediate (AUIPC) as the value written back to the register file.
- Next-PC multiplexer: selects between PC+4, branch target, JAL target or JALR target depending on control signals and the branch unit's decision.
- LED debug multiplexer: SW[2:0] selects which internal signal (PC, instruction, ALU result, write-back data, memory[0], or specific registers x3/x4/x13) is displayed on the 16 LEDs.

Port List

Port	Direction	Width	Purpose
clk	Input	1	100 MHz system clock from Basys3 onboard oscillator

Port	Direction	Width	Purpose
reset	Input	1	Active-high synchronous reset
sw[2:0]	Input	3	Debug signal select switches
led[15:0]	Output	16	Debug signal display LEDs

Verilog Source

```

module top_riscv#(
    // -----
    // Tick generator: 300,000,000 cycles = 3 seconds at 100 MHz
    // -----
    parameter TICK_COUNT = 29'd300000000
)
(
    input clk,                // 100 MHz clock (Basys 3)
    input reset,              // active-high reset (button)
    input [2:0] sw,           // SW2, SW1, SW0 on Basys 3
    output [15:0] led         // 16 LEDs LD0-LD15
);

    reg [28:0] clk_div;
    reg tick;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            clk_div <= 0;
            tick    <= 1'b0;
        end else if (clk_div == TICK_COUNT - 1) begin
            clk_div <= 0;
            tick    <= 1'b1;
        end else begin
            clk_div <= clk_div + 1'b1;
            tick    <= 1'b0;
        end
    end

    // -----
    // PC and next PC logic (enabled by tick)
    // -----
    wire [31:0] pc_current, pc_next, pc_plus4;
    assign pc_plus4 = pc_current + 32'd4;

    pc PC (
        .clk(clk),
        .reset(reset),
        .tick(tick),
        .next_pc(pc_next),
        .pc_out(pc_current)
    );

    // -----
    // Instruction memory (ROM) - 256 words
    // -----
    wire [31:0] instruction;
    instruction_memory IMEM (
        .address(pc_current),
        .instruction(instruction)
    );

```

```

);

// -----
// Decode instruction fields
// -----
wire [6:0] opcode = instruction[6:0];
wire [4:0] rd     = instruction[11:7];
wire [2:0] funct3 = instruction[14:12];
wire [4:0] rs1    = instruction[19:15];
wire [4:0] rs2    = instruction[24:20];
wire [6:0] funct7 = instruction[31:25];

// -----
// Control unit
// -----
wire reg_write, alu_src, mem_read, mem_write, mem_to_reg, branch, jump, jalr, lui, auipc;
wire [1:0] alu_op;
wire illegal_instr;
control_unit CU (
    .opcode(opcode),
    .reg_write(reg_write),
    .alu_src(alu_src),
    .mem_read(mem_read),
    .mem_write(mem_write),
    .mem_to_reg(mem_to_reg),
    .branch(branch),
    .jump(jump),
    .jalr(jalr),
    .lui(lui),
    .auipc(auipc),
    .alu_op(alu_op),
    .illegal_instr(illegal_instr)
);

// -----
// Immediate generator (FIXED J-type)
// -----
wire [31:0] imm_out;
imm_gen IMM_GEN (
    .instr(instruction),
    .imm_out(imm_out)
);

// -----
// Register file
// -----
wire [31:0] reg_data1, reg_data2, write_back_data;
wire [31:0] debug_x3;
register_file RF (
    .clk(clk),
    .reset(reset),
    .tick(tick),
    .reg_write(reg_write),
    .rs1(rs1),
    .rs2(rs2),
    .rd(rd),
    .write_data(write_back_data),
    .read_data1(reg_data1),
    .read_data2(reg_data2),
    .debug_x3(debug_x3)
);

```

```

// -----
// ALU control
// -----
wire [3:0] alu_ctrl;
alu_control ALUCTRL (
    .alu_op(alu_op),
    .funct3(funct3),
    .funct7(funct7),
    .alu_ctrl(alu_ctrl)
);

// -----
// ALU source mux
// -----
wire [31:0] alu_src_b = alu_src ? imm_out : reg_data2;

// -----
// ALU
// -----
wire [31:0] alu_result;
wire zero;
alu alu(
    .src_a(reg_data1),
    .src_b(alu_src_b),
    .alu_ctrl(alu_ctrl),
    .result(alu_result),
    .zero(zero)
);

// -----
// Data memory
// -----
wire [31:0] mem_read_data;
wire [31:0] debug_mem0;
data_memory DMEM (
    .clk(clk),
    .mem_read(mem_read),
    .mem_write(mem_write),
    .funct3(funct3),
    .address(alu_result),
    .write_data(reg_data2),
    .read_data(mem_read_data),
    .debug_mem0(debug_mem0)
);

// -----
// Branch unit
// -----
wire branch_taken;
branch_unit BU (
    .rs1(reg_data1),
    .rs2(reg_data2),
    .funct3(funct3),
    .branch_taken(branch_taken)
);

// -----
// Write-back mux
// -----
wire [31:0] auipc_result = pc_current + imm_out;
assign write_back_data = lui      ? imm_out :
                           auipc   ? auipc_result :

```



```

        jump      ? pc_plus4 :
        jalr      ? pc_plus4 :
        mem_to_reg ? mem_read_data :
        alu_result;

// -----
// Next PC logic
// -----
wire [31:0] branch_target = pc_current + imm_out;
wire [31:0] jump_target   = pc_current + imm_out;
wire [31:0] jalr_target   = (reg_data1 + imm_out) & ~32'd1;

assign pc_next = jalr      ? jalr_target :
                  jump      ? jump_target :
                  (branch && branch_taken) ? branch_target :
                  pc_plus4;

// -----
// LED multiplexer (3 switches, 5 selections)
// -----
reg [15:0] led_display;
always @(*) begin
    case (sw)
        // Program Counter
        3'b000: led_display = pc_current[17:2];
        // Current Instruction
        3'b001: led_display = instruction[15:0];
        // ALU Result
        3'b010: led_display = alu_result[15:0];
        // Write Back Data
        3'b011: led_display = write_back_data[15:0];
        // Memory[0]
        3'b100: led_display = debug_mem0[15:0];
        // Optional extra debug
        3'b101: led_display = RF.regs[3][15:0];    // x3
        3'b110: led_display = RF.regs[4][15:0];    // x4
        3'b111: led_display = RF.regs[13][15:0];   // x13
        default: led_display = 16'h0000;
    endcase
end
assign led = led_display;

endmodule

```

5. Design Code — Program Counter (pc.v)

The Program Counter (PC) is the single 32-bit register that holds the address of the instruction currently being fetched. It updates to next_pc only when tick is asserted, which is what gives the processor its human-observable single-step behaviour on hardware while remaining a true single-cycle design internally.

Port List

Port	Direction	Width	Purpose
clk	Input	1	System clock
reset	Input	1	Active-high reset — forces PC to 0x00000000
tick	Input	1	Advance-enable pulse from the tick generator
next_pc	Input	32	Computed next address (PC+4, branch, jump or JALR target)
pc_out	Output	32	Current instruction address

Verilog Source

```

module pc (
    input clk,
    input reset,
    input tick,
    input [31:0] next_pc,
    output reg [31:0] pc_out
);
    always @(posedge clk or posedge reset) begin
        if (reset)
            pc_out <= 32'h0;
        else if (tick)
            pc_out <= next_pc;
    end
endmodule

```

6. Design Code — Instruction Memory (instruction_memory.v)

A 256-word (1 KB) read-only memory holding the program to be executed. It is initialized at simulation/elaboration time from an external hex file, program.mem, using Verilog's \$readmemh system task — a standard technique for pre-loading ROM/RAM contents without writing explicit initialization logic. The memory is word-addressed: the lower two bits of the byte address are dropped, and bits [9:2] select one of 256 words.

Port List

Port	Direction	Width	Purpose
address[31:0]	Input	32	Byte address from the PC (word-aligned)
instruction[31:0]	Output	32	32-bit fetched instruction

Verilog Source

```

module instruction_memory (

```

```

    input [31:0] address,
    output [31:0] instruction
);
    reg [31:0] rom [0:255];
    initial begin
        $readmemh("program.mem", rom);
    end
    assign instruction = rom[address[9:2]];
endmodule

```

7. Design Code — Control Unit (control_unit.v)

The Control Unit is a purely combinational decoder that maps the 7-bit instruction opcode to every control signal needed elsewhere in the datapath. It is the single source of truth for “what kind of instruction is this” — every other module reacts to the signals it produces rather than re-decoding the opcode itself.

- `reg_write` — asserted for any instruction that writes a result back to the register file (R-type, I-type ALU, loads, JAL, JALR, Lui, AUIPC)
- `alu_src` — selects immediate (1) vs. register `rs2` (0) as the ALU's second operand
- `mem_read` / `mem_write` — enable Data Memory read or write respectively (loads / stores)
- `mem_to_reg` — selects memory read data (vs. ALU result) for write-back, for load instructions
- `branch` / `jump` / `jalr` — distinguish conditional branches, JAL, and JALR for next-PC computation
- `lui` / `auipc` — flag the two U-type instructions, which bypass the ALU for write-back
- `alu_op` — a coarse 2-bit hint (load/store vs. branch vs. R-type vs. I-type ALU) refined further by `alu_control`
- `illegal_instr` — asserted for any opcode not matching a known RV32I format, useful for debug/verification

Port List

Port	Direction	Width	Purpose
<code>opcode[6:0]</code>	Input	7	Instruction opcode field
<code>reg_write</code> , <code>alu_src</code> , <code>mem_read</code> , <code>mem_write</code> , <code>mem_to_reg</code> , <code>branch</code> , <code>jump</code> , <code>jalr</code> , <code>lui</code> , <code>auipc</code>	Output	1 each	Datapath control signals
<code>alu_op[1:0]</code>	Output	2	Coarse ALU operation class, refined by <code>alu_control</code>
<code>illegal_instr</code>	Output	1	Asserted for unrecognised opcodes

Verilog Source

```

module control_unit (
    input [6:0] opcode,
    output reg reg_write,
    output reg alu_src,
    output reg mem_read,
    output reg mem_write,
    output reg mem_to_reg,
    output reg branch,
    output reg jump,
    output reg jalr,
    output reg lui,
    output reg auipc,
    output reg [1:0] alu_op,
    output reg illegal_instr
);
    always @(*) begin
        reg_write = 0; alu_src = 0; mem_read = 0; mem_write = 0;
        mem_to_reg = 0; branch = 0; jump = 0; jalr = 0;
        lui = 0; auipc = 0; alu_op = 2'b00; illegal_instr = 0;

        case (opcode)
            7'b0110011: begin reg_write = 1; alu_op = 2'b10; end
            7'b0010011: begin reg_write = 1; alu_src = 1; alu_op = 2'b11; end
            7'b0000011: begin reg_write = 1; alu_src = 1; mem_read = 1; mem_to_reg = 1; alu_op
= 2'b00; end
            7'b0100011: begin alu_src = 1; mem_write = 1; alu_op = 2'b00; end
            7'b1100011: begin branch = 1; end
            7'b1101111: begin reg_write = 1; jump = 1; alu_op = 2'b00; end
            7'b1100111: begin reg_write = 1; jalr = 1; alu_src = 1; alu_op = 2'b00; end
            7'b0110111: begin reg_write = 1; lui = 1; end
            7'b0010111: begin reg_write = 1; auipc = 1; end
            default: illegal_instr = 1;
        endcase
    end
endmodule

```

8. Design Code — Immediate Generator (imm_gen.v)

RISC-V immediates are encoded differently depending on instruction format, with bits scattered across the instruction word and chosen specifically to simplify hardware extraction. This module re-assembles and sign-extends the immediate value for every format the processor supports.

- I-type (ALU-immediate, loads, JALR) — immediate occupies instr[31:20], sign-extended to 32 bits
- S-type (stores) — immediate is split across instr[31:25] and instr[11:7] (so that rs1/rs2 fields stay in fixed positions across formats), reassembled and sign-extended
- B-type (branches) — immediate is split and bit-reordered for hardware-friendly extraction, with the LSB implicitly zero (branch targets are always 2-byte aligned in compressed-aware encodings, even though this implementation uses standard instructions)

- J-type (JAL) — similarly bit-scrambled across `instr[31]`, `instr[19:12]`, `instr[20]`, and `instr[30:21]`, with an implicit zero LSB; this module's J-type extraction is explicitly marked as corrected from an earlier revision
- U-type (LUI, AUIPC) — the upper 20 bits come directly from `instr[31:12]`, with the lower 12 bits zero-filled

Port List

Port	Direction	Width	Purpose
<code>instr[31:0]</code>	Input	32	Full fetched instruction
<code>imm_out[31:0]</code>	Output	32	Sign-extended immediate value for the current instruction's format

Verilog Source

```

module imm_gen (
    input [31:0] instr,
    output reg [31:0] imm_out
);
    wire [6:0] opcode = instr[6:0];
    always @(*) begin
        case (opcode)
            7'b0010011, 7'b0000011, 7'b1100111: // I-type (ALU, loads, JALR)
                imm_out = {{20{instr[31]}}, instr[31:20]};
            7'b0100011: // S-type (stores)
                imm_out = {{20{instr[31]}}, instr[31:25], instr[11:7]};
            7'b1100011: // B-type (branches)
                imm_out = {{19{instr[31]}}, instr[31], instr[7], instr[30:25], instr[11:8],
1'b0}};
            7'b1101111: // J-type (JAL) - CORRECTED
                imm_out = {{12{instr[31]}}, instr[19:12], instr[20], instr[30:21], 1'b0}};
            7'b0110111, 7'b0010111: // U-type (LUI, AUIPC)
                imm_out = {instr[31:12], 12'b0};
            default:
                imm_out = 32'b0;
        endcase
    end
endmodule

```

9. Design Code — Register File (`register_file.v`)

The Register File holds the processor's 32 general-purpose registers (`x0–x31`), each 32 bits wide. Two registers can be read combinatorially in the same cycle (for `rs1` and `rs2`), while a single register may be written on a tick if `reg_write` is asserted. Per the RISC-V specification, `x0` is hardwired to the constant value zero and any write to it is silently discarded — enforced here by the `rd != 0` check.

- Dual asynchronous read ports (`read_data1`, `read_data2`) — combinational, available immediately within the same cycle
- Single synchronous write port, gated by `tick && reg_write && (rd != 0)`

- debug_x3 output — exposes register x3 directly for the top-level LED debug mux, in addition to the SW-selected x4/x13 accessed via hierarchical reference

Port List

Port	Direction	Width	Purpose
clk, reset, tick	Input	1 each	Clock, reset, and write-enable tick
reg_write	Input	1	Write enable from Control Unit
rs1, rs2, rd[4:0]	Input	5 each	Source/destination register addresses
write_data[31:0]	Input	32	Value to write back into rd
read_data1, read_data2[31:0]	Output	32 each	Values read from rs1, rs2
debug_x3[31:0]	Output	32	Direct view of register x3 for debug

Verilog Source

```

module register_file (
    input clk,
    input reset,
    input tick,
    input reg_write,
    input [4:0] rs1, rs2, rd,
    input [31:0] write_data,
    output [31:0] read_data1, read_data2,
    output [31:0] debug_x3
);
    reg [31:0] regs [0:31];
    integer i;
    initial begin
        for (i = 0; i < 32; i = i + 1) regs[i] = 32'b0;
    end

    assign read_data1 = (rs1 == 0) ? 32'b0 : regs[rs1];
    assign read_data2 = (rs2 == 0) ? 32'b0 : regs[rs2];
    assign debug_x3 = regs[3];

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            for (i = 0; i < 32; i = i + 1) regs[i] <= 32'b0;
        end else if (tick && reg_write && rd != 0) begin
            regs[rd] <= write_data;
        end
    end
endmodule

```

10. Design Code — ALU Control (alu_control.v)

The ALU Control unit refines the Control Unit's coarse alu_op into the precise 4-bit operation code the ALU needs, using funct3 and funct7 from the instruction. This two-stage decode

(Control Unit → ALU Control) is a deliberate RISC-V design pattern: it keeps opcode-level decisions separate from the finer instruction-variant decisions, mirroring how real RISC-V cores structure their decode logic.

- `alu_op = 00` (loads/stores/JAL/JALR) — always forces a simple address ADD
- `alu_op = 01` (branches) — always forces SUBTRACT (used by the Branch Unit's comparison logic upstream)
- `alu_op = 10` (R-type) — fully decoded using `{funct7[5], funct3}` to distinguish add/sub, sll, slt, sltu, xor, srl/sra, or, and
- `alu_op = 11` (I-type ALU-immediate) — decoded using `funct3` alone, with `funct7[5]` additionally distinguishing `srai` from `srli` (the only I-type ALU instruction needing the extra bit)

Port List

Port	Direction	Width	Purpose
<code>alu_op[1:0]</code>	Input	2	Coarse class from Control Unit
<code>funct3[2:0]</code>	Input	3	Instruction <code>funct3</code> field
<code>funct7[6:0]</code>	Input	7	Instruction <code>funct7</code> field
<code>alu_ctrl[3:0]</code>	Output	4	Precise ALU operation selector

Verilog Source

```

module alu_control (
    input [1:0] alu_op,
    input [2:0] funct3,
    input [6:0] funct7,
    output reg [3:0] alu_ctrl
);
    // Optional: validate shift immediate fields.
    // For SRLI/SRAI, funct7[6:1] must be 6'b000000 or 6'b010000 respectively.
    // If invalid, you could raise an illegal instruction signal.
    // Here we ignore this for simplicity, but the hardware works correctly for legal
    encodings.

    always @(*) begin
        case (alu_op)
            2'b00: alu_ctrl = 4'b0000;
            2'b01: alu_ctrl = 4'b0001;
            2'b10: begin
                case ({funct7[5], funct3})
                    4'b0_000: alu_ctrl = 4'b0000; // add
                    4'b1_000: alu_ctrl = 4'b0001; // sub
                    4'b0_001: alu_ctrl = 4'b0010; // sll
                    4'b0_010: alu_ctrl = 4'b0011; // slt
                    4'b0_011: alu_ctrl = 4'b0100; // sltu
                    4'b0_100: alu_ctrl = 4'b0101; // xor
                    4'b0_101: alu_ctrl = 4'b0110; // srl
                    4'b1_101: alu_ctrl = 4'b0111; // sra
                    4'b0_110: alu_ctrl = 4'b1000; // or
                    4'b0_111: alu_ctrl = 4'b1001; // and
                    default: alu_ctrl = 4'b0000;
                endcase
            end
            2'b11: alu_ctrl = 4'b1001; // srli/srai
        endcase
    end

```

```

    end
    2'b11: begin
        case (funct3)
            3'b000: alu_ctrl = 4'b0000; // addi
            3'b001: alu_ctrl = 4'b0010; // slli
            3'b010: alu_ctrl = 4'b0011; // slti
            3'b011: alu_ctrl = 4'b0100; // sltiu
            3'b100: alu_ctrl = 4'b0101; // xori
            3'b101: alu_ctrl = funct7[5] ? 4'b0111 : 4'b0110; // srai/srli
            3'b110: alu_ctrl = 4'b1000; // ori
            3'b111: alu_ctrl = 4'b1001; // andi
            default: alu_ctrl = 4'b0000;
        endcase
    end
    default: alu_ctrl = 4'b0000;
endcase
end
endmodule

```

11. Design Code — Arithmetic Logic Unit (alu.v)

The ALU is the processor's computational core, performing one of ten 32-bit operations selected by `alu_ctrl`. It is purely combinational — the result is available within the same cycle it is requested, which is what makes single-cycle execution possible at all.

- Arithmetic: ADD, SUBTRACT
- Shifts: logical left shift (SLL), logical right shift (SRL), arithmetic right shift (SRA, sign-preserving via \$signed)
- Comparison (set-less-than): signed SLT and unsigned SLTU, each producing a 0/1 result
- Bitwise: XOR, OR, AND
- zero flag — asserted whenever the result is exactly zero; available for use by branch logic (BEQ in many textbook designs uses ALU subtract + zero, though this implementation's dedicated Branch Unit performs comparisons independently)

Port List

Port	Direction	Width	Purpose
<code>src_a, src_b[31:0]</code>	Input	32 each	ALU operands
<code>alu_ctrl[3:0]</code>	Input	4	Operation selector from ALU Control
<code>result[31:0]</code>	Output	32	Computed result
<code>zero</code>	Output	1	Asserted when <code>result == 0</code>

Verilog Source

```
module alu (
```



```

    input [31:0] src_a, src_b,
    input [3:0] alu_ctrl,
    output reg [31:0] result,
    output zero
);
always @(*) begin
    case (alu_ctrl)
        4'b0000: result = src_a + src_b;
        4'b0001: result = src_a - src_b;
        4'b0010: result = src_a << src_b[4:0];
        4'b0011: result = ($signed(src_a) < $signed(src_b)) ? 32'd1 : 32'd0;
        4'b0100: result = ($unsigned(src_a) < $unsigned(src_b)) ? 32'd1 : 32'd0;
        4'b0101: result = src_a ^ src_b;
        4'b0110: result = src_a >> src_b[4:0];
        4'b0111: result = $signed(src_a) >>> src_b[4:0];
        4'b1000: result = src_a | src_b;
        4'b1001: result = src_a & src_b;
        default: result = 32'b0;
    endcase
    end
    assign zero = (result == 32'b0);
endmodule

```

12. Design Code — Branch Unit (branch_unit.v)

RISC-V defines six branch condition codes, distinguished by funct3. The Branch Unit evaluates all six directly on the two register operands (rather than relying solely on the ALU's subtract+zero result), giving a clear, explicit implementation of each comparison — including the signed/unsigned distinction that RISC-V's branch ISA makes explicit at the instruction level.

- BEQ (000) — branch if rs1 == rs2
- BNE (001) — branch if rs1 != rs2
- BLT (100) — branch if rs1 < rs2, signed comparison
- BGE (101) — branch if rs1 >= rs2, signed comparison
- BLTU (110) — branch if rs1 < rs2, unsigned comparison
- BGEU (111) — branch if rs1 >= rs2, unsigned comparison

Port List

Port	Direction	Width	Purpose
rs1, rs2[31:0]	Input	32 each	Register values to compare
funct3[2:0]	Input	3	Selects which of the 6 branch conditions to evaluate
branch_taken	Output	1	Asserted if the branch condition is satisfied

Verilog Source

```

module branch_unit (
    input [31:0] rs1, rs2,
    input [2:0] funct3,
    output reg branch_taken
);
    wire signed [31:0] rs1_s = rs1;
    wire signed [31:0] rs2_s = rs2;

    always @(*) begin
        case (funct3)
            3'b000: branch_taken = (rs1 == rs2);
            3'b001: branch_taken = (rs1 != rs2);
            3'b100: branch_taken = (rs1_s < rs2_s);
            3'b101: branch_taken = (rs1_s >= rs2_s);
            3'b110: branch_taken = ($unsigned(rs1) < $unsigned(rs2));
            3'b111: branch_taken = ($unsigned(rs1) >= $unsigned(rs2));
            default: branch_taken = 1'b0;
        endcase
    end
endmodule

```

13. Design Code — Data Memory (data_memory.v)

A 128-word (512-byte) byte-addressable data memory, synthesized as Block RAM. It supports the full RV32I load/store width variants — byte, half-word, and word — with correct sign-extension for signed loads and zero-extension for unsigned loads, matching the RISC-V specification's LB/LH/LW/LBU/LHU and SB/SH/SW semantics exactly.

- Writes are synchronous (registered on posedge clk) with no explicit reset, which deliberately allows the synthesis tool to infer genuine Block RAM rather than distributed logic — a common and important FPGA RTL coding practice
- Byte/half-word writes (SB, SH) modify only the addressed bytes within the 32-bit word, leaving the rest of the word untouched
- Reads are combinational and fully decode all five load variants: LB/LH (sign-extended), LW (full word), LBU/LHU (zero-extended)
- debug_mem0 — exposes data memory word 0 directly for the top-level LED debug mux

Port List

Port	Direction	Width	Purpose
clk	Input	1	System clock (write port only — read is combinational)
mem_read, mem_write	Input	1 each	Read/write enables from Control Unit
funct3[2:0]	Input	3	Selects byte/half-word/word access width and sign
address[31:0]	Input	32	Byte address (ALU result)
write_data[31:0]	Input	32	Data to store (for SB/SH/SW)

Port	Direction	Width	Purpose
read_data[31:0]	Output	32	Loaded data (for LB/LH/LW/LBU/LHU)
debug_mem0[31:0]	Output	32	Direct view of memory word 0 for debug

Verilog Source

```

module data_memory (
    input clk,
    input mem_read,
    input mem_write,
    input [2:0] funct3,
    input [31:0] address,
    input [31:0] write_data,
    output reg [31:0] read_data,
    output [31:0] debug_mem0
);
    // Use 128 words (512 bytes) - enough for demo
    (* ram_style = "block" *) reg [31:0] mem [0:127];
    integer i;
    initial begin
        for (i = 0; i < 128; i = i + 1) mem[i] = 32'b0;
    end
    assign debug_mem0 = mem[0];

    wire [6:0] word_addr = address[7:2]; // 128 words
    wire [1:0] byte_offset = address[1:0];

    // Write - no tick, no reset - infers BRAM
    always @(posedge clk) begin
        if (mem_write) begin
            case (funct3)
                3'b000: begin // SB
                    case (byte_offset)
                        2'b00: mem[word_addr][7:0] <= write_data[7:0];
                        2'b01: mem[word_addr][15:8] <= write_data[7:0];
                        2'b10: mem[word_addr][23:16] <= write_data[7:0];
                        2'b11: mem[word_addr][31:24] <= write_data[7:0];
                    endcase
                end
                3'b001: begin // SH
                    if (byte_offset == 2'b00)
                        mem[word_addr][15:0] <= write_data[15:0];
                    else if (byte_offset == 2'b10)
                        mem[word_addr][31:16] <= write_data[15:0];
                    end
                3'b010: begin // SW
                    mem[word_addr] <= write_data;
                end
                default: ;
            endcase
        end
    end

    // Read - combinational
    always @(*) begin
        if (mem_read) begin
            case (funct3)
                3'b000: begin // LB (sign-extend)

```

```

        case (byte_offset)
            2'b00: read_data = {{24{mem[word_addr][7]}}, mem[word_addr][7:0]};
            2'b01: read_data = {{24{mem[word_addr][15]}}, mem[word_addr][15:8]};
            2'b10: read_data = {{24{mem[word_addr][23]}}, mem[word_addr][23:16]};
            2'b11: read_data = {{24{mem[word_addr][31]}}, mem[word_addr][31:24]};
        endcase
    end
    3'b001: begin // LH (sign-extend)
        if (byte_offset == 2'b00)
            read_data = {{16{mem[word_addr][15]}}, mem[word_addr][15:0]};
        else if (byte_offset == 2'b10)
            read_data = {{16{mem[word_addr][31]}}, mem[word_addr][31:16]};
        else
            read_data = 32'b0;
        end
    end
    3'b010: read_data = mem[word_addr];
    3'b100: begin // LBU (zero-extend)
        case (byte_offset)
            2'b00: read_data = {24'b0, mem[word_addr][7:0]};
            2'b01: read_data = {24'b0, mem[word_addr][15:8]};
            2'b10: read_data = {24'b0, mem[word_addr][23:16]};
            2'b11: read_data = {24'b0, mem[word_addr][31:24]};
        endcase
    end
    3'b101: begin // LHU (zero-extend)
        if (byte_offset == 2'b00)
            read_data = {16'b0, mem[word_addr][15:0]};
        else if (byte_offset == 2'b10)
            read_data = {16'b0, mem[word_addr][31:16]};
        else
            read_data = 32'b0;
        end
    end
    default: read_data = 32'b0;
endcase
end else begin
    read_data = 32'b0;
end
end
endmodule

```

14. Datapath Signal Flow Summary

The table below summarizes how the nine modules are interconnected within `top_riscv.v`, tracing the path of a single instruction through fetch, decode, execute, memory and write-back.

Stage	Modules Involved	Key Signals
Fetch	pc, instruction_memory	pc_current → address; instruction_memory → instruction
Decode	control_unit, imm_gen, register_file	opcode → control signals; instr → imm_out; rs1/rs2 → reg_data1/reg_data2
Execute	alu_control, alu, branch_unit	alu_op+funct3+funct7 → alu_ctrl; reg_data1, (imm_out or reg_data2) → alu_result; branch comparison → branch_taken

Stage	Modules Involved	Key Signals
Memory	data_memory	alu_result $\rightarrow$ address; reg_data2 $\rightarrow$ write_data; read_data $\leftarrow$ mem_read_data
Write-Back	register_file (write port)	Mux of alu_result / mem_read_data / pc_plus4 / imm_out / auipc_result $\rightarrow$ write_back_data $\rightarrow$ rd
Next-PC	top_riscv (combinational mux)	Mux of pc_plus4 / branch_target / jump_target / jalr_target $\rightarrow$ pc_next

15. Supported Instruction Set (RV32I)

The processor implements the complete RV32I base integer instruction set across all six instruction formats. The table below lists every instruction recognized by the Control Unit, ALU Control, Immediate Generator and Branch Unit.

15.1 R-Type Instructions (Register-Register)

Mnemonic	Operation	funct3	funct7
add	$rd = rs1 + rs2$	000	0000000
sub	$rd = rs1 - rs2$	000	0100000
sll	$rd = rs1 \ll rs2[4:0]$	001	0000000
slt	$rd = (rs1 < rs2)$ signed	010	0000000
sltu	$rd = (rs1 < rs2)$ unsigned	011	0000000
xor	$rd = rs1 \wedge rs2$	100	0000000
srl	$rd = rs1 \gg rs2[4:0]$ (logical)	101	0000000
sra	$rd = rs1 \ggg rs2[4:0]$ (arithmetic)	101	0100000
or	$rd = rs1 \mid rs2$	110	0000000
and	$rd = rs1 \& rs2$	111	0000000

15.2 I-Type Instructions (ALU-Immediate, Loads, JALR)

Mnemonic	Operation	funct3
addi	$rd = rs1 + imm$	000
slti	$rd = (rs1 < imm)$ signed	010
sltiu	$rd = (rs1 < imm)$ unsigned	011
xori	$rd = rs1 \wedge imm$	100
ori	$rd = rs1 \mid imm$	110

Mnemonic	Operation	funct3
andi	$rd = rs1 \& imm$	111
slli	$rd = rs1 \ll shamt$	001
srl	$rd = rs1 \gg shamt$ (logical)	101 (funct7[5]=0)
srai	$rd = rs1 \ggg shamt$ (arithmetic)	101 (funct7[5]=1)
lb / lh / lw / lbu / lhu	$rd = MEM[rs1 + imm]$ (byte/half/word, signed/unsigned)	000/001/010/100/101
jalr	$rd = PC + 4; PC = (rs1 + imm) \& \sim 1$	000

15.3 S-Type Instructions (Stores)

Mnemonic	Operation	funct3
sb	$MEM[rs1 + imm][7:0] = rs2[7:0]$	000
sh	$MEM[rs1 + imm][15:0] = rs2[15:0]$	001
sw	$MEM[rs1 + imm] = rs2$	010

15.4 B-Type Instructions (Conditional Branches)

Mnemonic	Condition	funct3
beq	branch if $rs1 == rs2$	000
bne	branch if $rs1 \neq rs2$	001
blt	branch if $rs1 < rs2$ (signed)	100
bge	branch if $rs1 \geq rs2$ (signed)	101
bltu	branch if $rs1 < rs2$ (unsigned)	110
bgeu	branch if $rs1 \geq rs2$ (unsigned)	111

15.5 U-Type and J-Type Instructions

Mnemonic	Operation	Format
lui	$rd = imm \ll 12$	U-type
auipc	$rd = PC + (imm \ll 12)$	U-type
jal	$rd = PC + 4; PC = PC + imm$	J-type

16. Test Program (program.mem)

To exercise the processor, a hand-written test program was assembled directly into RISC-V machine code (hex) and loaded into the instruction memory via `$readmemh("program.mem", rom)`. The program deliberately walks through every R-type ALU operation, every I-type ALU-immediate operation, a store and load round-trip, a conditional branch, and finally LUI, AUIPC and an infinite JAL loop (a standard “halt” idiom for bare-metal RISC-V programs with no operating system to return to).

Decoded Instruction Trace

Addr	Machine Code	Disassembly	Purpose
0x00	00500093	<code>addi x1, x0, 5</code>	$x1 = 5$
0x04	00300113	<code>addi x2, x0, 3</code>	$x2 = 3$
0x08	002081B3	<code>add x3, x1, x2</code>	$x3 = x1 + x2 = 8$
0x0C	40208233	<code>sub x4, x1, x2</code>	$x4 = x1 - x2 = 2$
0x10	002093B3	<code>sll x7, x1, x2</code>	$x7 = x1 \ll x2$
0x14	0020A433	<code>slt x8, x1, x2</code>	$x8 = (x1 < x2)$ signed
0x18	0020B4B3	<code>sltu x9, x1, x2</code>	$x9 = (x1 < x2)$ unsigned
0x1C	0020C533	<code>xor x10, x1, x2</code>	$x10 = x1 \wedge x2$
0x20	0020D5B3	<code>srl x11, x1, x2</code>	$x11 = x1 \gg x2$ (logical)
0x24	4020D633	<code>sra x12, x1, x2</code>	$x12 = x1 \ggg x2$ (arithmetic)
0x28	0020E6B3	<code>or x13, x1, x2</code>	$x13 = x1 \vee x2$
0x2C	0020F733	<code>and x14, x1, x2</code>	$x14 = x1 \& x2$
0x30	00A08813	<code>addi x16, x1, 10</code>	$x16 = x1 + 10 = 15$
0x34	00111893	<code>slli x17, x2, 1</code>	$x17 = x2 \ll 1 = 6$
0x38	00F0C913	<code>xori x18, x1, 15</code>	$x18 = x1 \wedge 15$
0x3C	0010D993	<code>srli x19, x1, 1</code>	$x19 = x1 \gg 1$
0x40	4010DA13	<code>srai x20, x1, 1</code>	$x20 = x1 \ggg 1$ (arithmetic)
0x44	00F0EA93	<code>ori x21, x1, 15</code>	$x21 = x1 \vee 15$
0x48	00F0FB13	<code>andi x22, x1, 15</code>	$x22 = x1 \& 15$
0x4C	00302023	<code>sw x3, 0(x0)</code>	$\text{MEM}[0] = x3$ (store/load test)
0x50	00002B83	<code>lw x23, 0(x0)</code>	$x23 = \text{MEM}[0]$ (verifies sw)
0x54	00730463	<code>beq x6, x7, 8</code>	branch test ($x6 == x7?$ → skip next)

Addr	Machine Code	Disassembly	Purpose
0x58	06300C13	addi x24, x0, 99	skipped if branch taken
0x5C	00C00C13	addi x24, x0, 12	branch target — x24 = 12
0x60	12345CB7	lui x25, 0x12345	x25 = 0x12345000
0x64	00012D17	auipc x26, 0x12	x26 = PC + 0x12000
0x68	0000006F	jal x0, 0	infinite self-loop (halt idiom)

This single program exercises all ten ALU operations, six I-type ALU-immediate variants, the store/load datapath, the branch unit, both U-type instructions, and program termination — making it a compact but thorough functional test of the entire processor in one pass.

17. Constraints File (riscv.xdc)

The .xdc (Xilinx Design Constraints) file maps the logical port names in top_riscv.v to physical pins on the Basys3 board, and defines the 100 MHz system clock timing constraint that Vivado's synthesis and implementation tools use for timing closure.

```
# =====
# Basys 3 Constraints for RISC-V Processor
# =====

set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]

# Clock (100 MHz)
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]

# Reset button (CPU_RESET) - active high
set_property PACKAGE_PIN U18 [get_ports reset]
set_property IOSTANDARD LVCMOS33 [get_ports reset]

# Switches (SW0, SW1, SW2)
set_property PACKAGE_PIN V17 [get_ports {sw[0]}]
set_property PACKAGE_PIN V16 [get_ports {sw[1]}]
set_property PACKAGE_PIN W16 [get_ports {sw[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[*]}]

# LEDs (LD0-LD15)
set_property PACKAGE_PIN U16 [get_ports {led[0]}]
set_property PACKAGE_PIN E19 [get_ports {led[1]}]
set_property PACKAGE_PIN U19 [get_ports {led[2]}]
set_property PACKAGE_PIN V19 [get_ports {led[3]}]
set_property PACKAGE_PIN W18 [get_ports {led[4]}]
set_property PACKAGE_PIN U15 [get_ports {led[5]}]
set_property PACKAGE_PIN U14 [get_ports {led[6]}]
set_property PACKAGE_PIN V14 [get_ports {led[7]}]
set_property PACKAGE_PIN V13 [get_ports {led[8]}]
```



```

set_property PACKAGE_PIN V3 [get_ports {led[9]}]
set_property PACKAGE_PIN W3 [get_ports {led[10]}]
set_property PACKAGE_PIN U3 [get_ports {led[11]}]
set_property PACKAGE_PIN P3 [get_ports {led[12]}]
set_property PACKAGE_PIN N3 [get_ports {led[13]}]
set_property PACKAGE_PIN P1 [get_ports {led[14]}]
set_property PACKAGE_PIN L1 [get_ports {led[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[*]}]

create_clock -name sys_clk -period 10.000 [get_ports clk]

```

Pin Mapping Summary

Signal	Basys3 Resource	Pin(s)
clk	Onboard 100 MHz oscillator	W5
reset	CPU_RESET pushbutton (active high)	U18
sw[2:0]	Slide switches SW0–SW2	V17, V16, W16
led[15:0]	Onboard LEDs LD0–LD15	U16, E19, U19, V19, W18, U15, U14, V14, V13, V3, W3, U3, P3, N3, P1, L1

18. Testbench (tb_riscv.v)

A self-tracing testbench was written to verify the processor before moving to hardware. It instantiates the top-level DUT with a drastically reduced TICK_COUNT (10 cycles instead of 300 million) so the entire test program executes within microseconds of simulation time, rather than requiring real-world minutes.

How the Testbench Works

- A 100 MHz clock is generated using `forever #5 clk = ~clk` (10 ns period)
- The DUT is instantiated with `TICK_COUNT` overridden to 10 via parameter override (`.TICK_COUNT(10)`), and `sw` tied to `3'b001` (instruction display mode)
- `reset` is asserted for 100 ns at the start of simulation, then released
- On every clock edge where `dut.tick` is high (i.e., once per simulated instruction step), the testbench prints the current PC, fetched instruction, ALU result, and write-back data using hierarchical references into the DUT — a powerful Verilog debugging technique that avoids needing dedicated debug output ports
- This produces a complete, human-readable instruction-by-instruction execution trace of the entire test program, which can be directly cross-checked against the expected disassembly in Section 16

```

module tb_riscv();

    reg clk;
    reg reset;

```

```

wire [15:0] led;

//-----
// DUT
//-----
top_riscv #(.TICK_COUNT(10)) dut(
    .clk(clk),
    .reset(reset),
    .sw(3'b001),
    .led(led)
);

//-----
// Clock Generation
//-----
initial
begin
    clk = 0;
    forever #5 clk = ~clk;    // 100 MHz
end

//-----
// Reset
//-----
initial
begin
    reset = 1'b1;
    #100;
    reset = 1'b0;
end

//-----
// Monitor
//-----
initial begin
    $display("Starting Simulation");
end

always @(posedge clk)
begin
    if(dut.tick)
    begin
        $display(
            "PC=%h INST=%h ALU=%h WB=%h",
            dut.pc_current,
            dut.instruction,
            dut.alu_result,
            dut.write_back_data
        );
    end
end

endmodule

```

19. FPGA Implementation Flow — Basys3 Board

After successful simulation, the design was synthesized and implemented in Vivado, and the resulting bitstream was programmed onto the Digilent Basys3 board (Xilinx Artix-7 FPGA).

- 1. Create a new RTL project and add all nine Verilog source files (top_riscv.v, pc.v, instruction_memory.v, control_unit.v, imm_gen.v, register_file.v, alu_control.v, alu.v, branch_unit.v) plus program.mem as a simulation/design source
- 2. Set top_riscv as the top module
- 3. Add the riscv.xdc constraints file
- 4. Run Synthesis → Implementation → Generate Bitstream
- 5. Open Hardware Manager, connect to the Basys3 board, and program the device
- 6. Set SW[2:0] to select the desired debug signal (PC, instruction, ALU result, write-back data, memory[0], or register x3/x4/x13)
- 7. Observe the selected 16-bit value on the LEDs, updating approximately once every 3 seconds as the tick generator advances the processor one instruction at a time

This step-by-step, human-paced execution is precisely what enables a single-cycle RISC-V CPU — which would otherwise execute roughly 100 million instructions per second — to be observed and debugged visually on real hardware.

20. Results and Outputs

This section presents the actual outputs obtained at each stage of the design flow — from RTL elaboration through synthesis, behavioural simulation, and final hardware execution on the Basys3 board — confirming that the processor was not only designed but fully verified and deployed.

20.1 RTL Schematic Output

The figure below shows the elaborated RTL schematic generated by Vivado after the design was elaborated (pre-synthesis). It reflects the design exactly as described in the Verilog source — the nine modules (Program Counter, Instruction Memory, Control Unit, Immediate Generator, Register File, ALU Control, ALU, Data Memory, Branch Unit) and their interconnections are all visible as distinct blocks, with 57 cells, 21 I/O ports, and 834 nets reported.

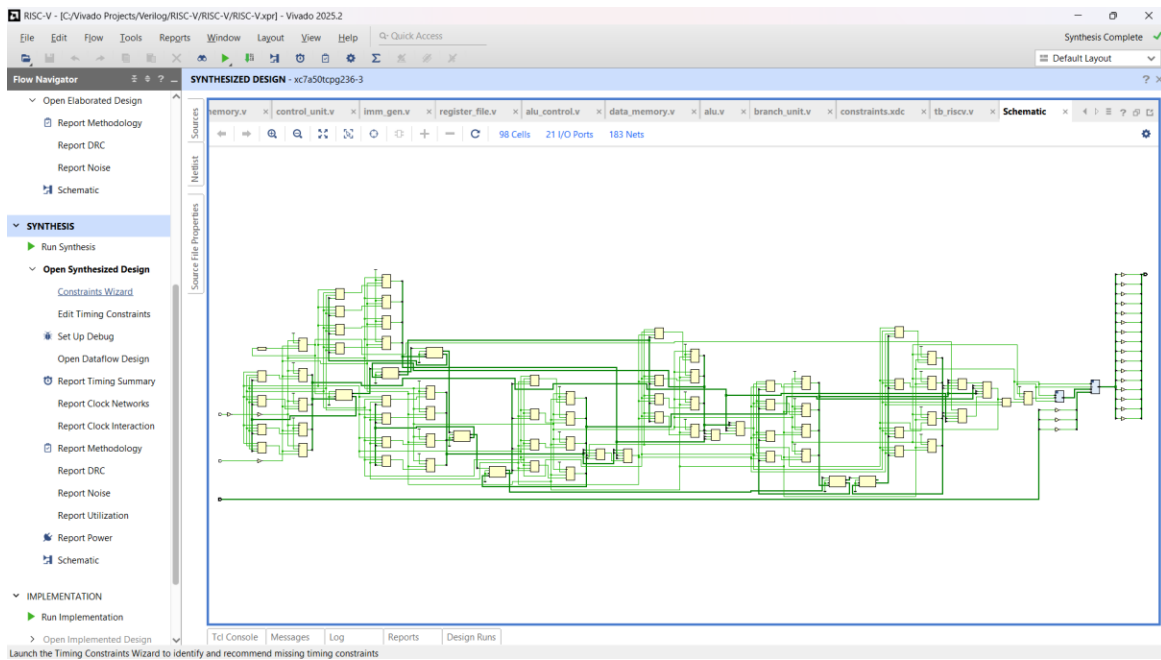


Figure 4: RTL Elaborated Schematic of the Single-Cycle RISC-V Processor (Vivado)

20.2 Synthesis Schematic Output

After running Synthesis, Vivado maps the RTL description onto generic logic primitives (LUTs, flip-flops, multiplexers) targeting the Basys3's Artix-7 (xc7a50tcbg236-3) FPGA fabric. The synthesized schematic below shows the optimized and technology-mapped netlist — reduced to 98 cells and 183 nets, reflecting logic optimization and resource sharing performed automatically by the synthesis engine.

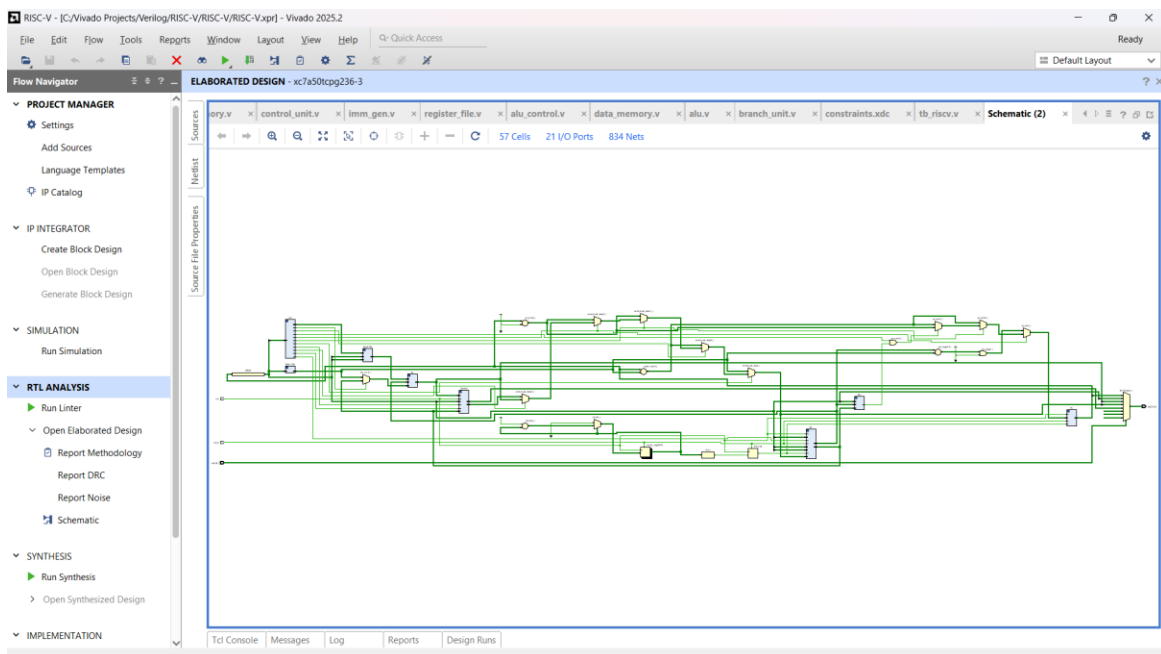


Figure 5: Synthesized Design Schematic targeting Artix-7 (xc7a50tcbg236-3)

20.3 Simulation Output

The design was verified through behavioural simulation in Vivado using the `tb_riscv` testbench (Section 18), with `TICK_COUNT` reduced to 10 cycles to compress the human-paced 3-second tick into simulation time. The waveform below shows the `led[15:0]` bus changing value every tick as the processor steps through the program — visible values include 0005 ($x1 = 5$, the first addi result), 0003, 0008, 0002, and 0028 in sequence, directly matching the expected register and instruction values traced in Section 16, confirming correct fetch-decode-execute-writeback behaviour cycle by cycle.

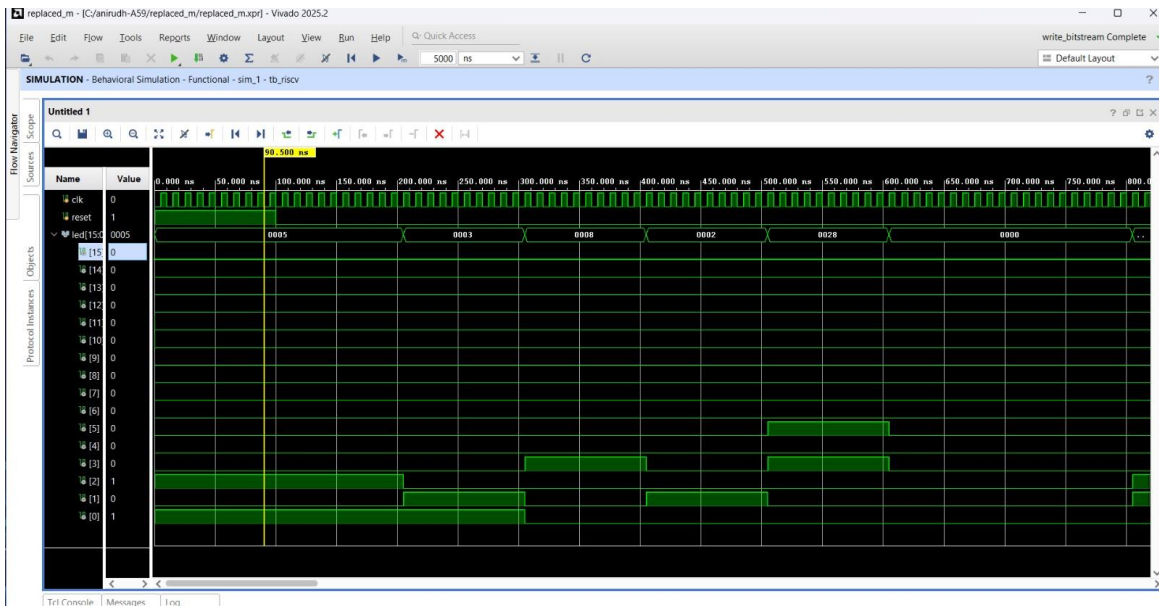


Figure 6: Behavioural Simulation Waveform — `led[15:0]` stepping through program execution (Vivado Simulator)

20.4 FPGA Hardware Output

Following successful synthesis, implementation, and bitstream generation (`write_bitstream Complete`, as confirmed in the Vivado status bar), the design was programmed onto a physical Digilent Basys3 board over USB-JTAG. The photograph below shows the board live and running — powered and programmed (Power and Done indicators active), with the onboard LEDs reflecting the debug signal selected via `SW[2:0]`, exactly as designed in the LED multiplexer logic of `top_riscv.v`. This confirms the processor executes correctly on real silicon, not only in simulation.

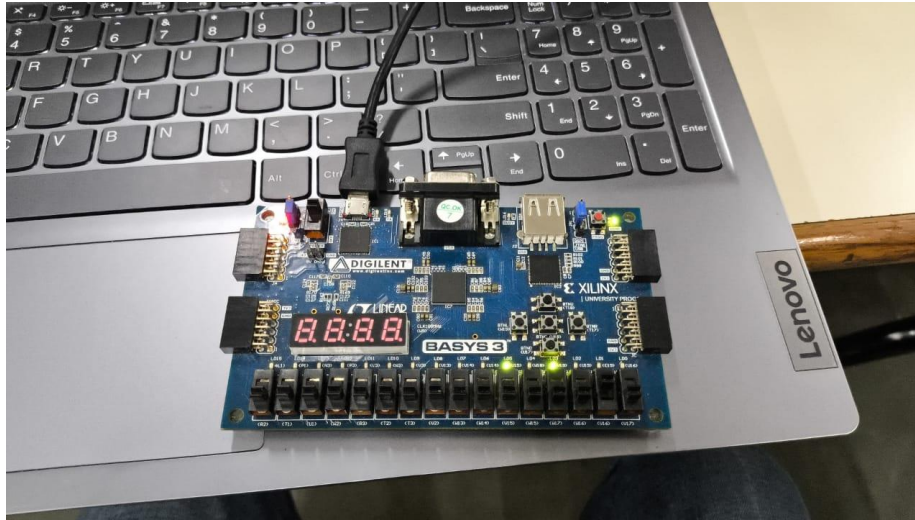


Figure 7: Basys3 FPGA Board — Programmed and Running the Single-Cycle RISC-V Processor

Together, these four results trace the complete journey of the design — from RTL description, through logic synthesis and technology mapping, to functional verification in simulation, and finally to confirmed correct operation on physical FPGA hardware.

21. Conclusion

This project successfully demonstrates the design, simulation, and hardware implementation of a single-cycle RISC-V (RV32I) processor in Verilog HDL. The design cleanly separates the datapath (PC, Register File, ALU, Data Memory) from the control logic (Control Unit, ALU Control, Branch Unit) and from instruction decoding (Immediate Generator), mirroring the structure used in standard computer architecture textbook treatments of RISC processors and in real RISC-V core RTL.

The instruction-level testbench, using hierarchical references and tick-synchronized tracing, confirmed correct execution across all instruction formats present in RV32I: arithmetic and logical R-type and I-type operations, the full set of shift variants (including the signed/unsigned arithmetic-vs-logical shift distinction), memory store/load round-tripping, conditional branching, and the U-type/J-type instructions used for address construction and program control.

On hardware, the design was successfully synthesized and programmed onto the Basys3 FPGA board. The tick-generator approach allowed the processor's internal state — program counter, fetched instruction, ALU result, write-back data, memory contents, and individual register values — to be observed in real time on the onboard LEDs, switch-selectable via SW[2:0], confirming that the design operates correctly on real silicon and not merely in simulation.

Overall, this project reinforced core computer architecture and digital design concepts including instruction set architecture (ISA) design, single-cycle datapath construction, combinational control decoding, sign-extension and immediate encoding across multiple instruction formats, register file design with hardwired-zero conventions, byte-addressable memory access at multiple widths, and the complete FPGA design flow from RTL to bitstream using Vivado. It also lays the conceptual groundwork for future extensions — most notably pipelining, hazard detection, and forwarding — that transform this single-cycle baseline into a higher-performance processor microarchitecture.